Members:

Kurt Martin Millares	**Streamlit Developer**
Jeremy Prince Millan	**KNIME workflow Developer**
Tyrael Lance Peralta	**Python Developer**

GROUP 21

Course Pathway Advising

Decision Support / Rule-Based Advising

Scenario: An academic adviser wants to recommend a course sequence using completed courses, prerequisites, interests, and career goals.

System question: How should the available records be ranked or organized to produce: Recommended course sequence and unmet prerequisites?

Implementation scope	Use prerequisite rules and weighted scoring
Primary algorithm	Rule-Based Reasoning
Compare with	Weighted Scoring
System input	Completed courses, prerequisites, interests, and target career
System output	Recommended course sequence and unmet prerequisites
Python method	import pandas as pd; import numpy as np

Part 1 Understand the Dataset

Class activity: Identify reference-only fields, analytical inputs, any required target label, the generated output, missing information, and one privacy, fairness, safety, or accessibility concern.

Part 2 Prepare and Check the Dataset

Minimum requirement: Create approximately 30 to 100 records using four to eight relevant input features. Validate ranges and categories, document every rule or weight, retain identifiers for reporting, and define a transparent tie-breaking rule.

Part 3 Build the KNIME Workflow

Suggested workflow: CSV Reader -> Column Filter -> Missing Value -> Rule Engine -> Math Formula -> Normalizer -> Sorter -> Top k

Evaluation: ranking consistency, sensitivity to rules or weights, validity, and explainability

Part 4 Recreate the Process in Python

Required process: Use import pandas as pd; import numpy as np. Load the CSV with pandas, apply the documented rules or weights, calculate scores, sort the records, handle ties, and export the ranked results.

Part 5 Create the Streamlit Interface

Interface requirement: Collect or upload these inputs: Completed courses, prerequisites, interests, and target career. Validate incomplete or invalid values, run the same Python process, and display recommended course sequence and unmet prerequisites with a brief explanation.

Part 6 Test Explain and Reflect

Testing: Test normal, missing, boundary, and uncertain cases. Compare the primary and comparison algorithms using one results table or appropriate chart.

Critical-thinking questions:

1. Which criterion or rule should receive the highest priority?

- Prerequisite completion should receive the highest priority because a student should not be advised to take an advanced course without completing its required foundation courses.

2. What should happen when no option satisfies every requirement?

- The system should recommend the closest valid pathway, identify the missing prerequisites, and provide a preparation sequence instead of returning no result.

3. How should ties, conflicts, or human overrides be handled and explained?

Ties should be resolved using this order:

1. Fewer unmet prerequisites.
2. Stronger target-career alignment.
3. Higher academic performance.
4. Alphabetical course order as the final transparent tie-breaker.

Human advisers should also be able to override the recommendation, but the system must require an explanation for the override.

Interpretation: Write a short explanation of the results and the project's limitations.

Project limitations

- The recommendations depend on the accuracy of the rules.
- Synthetic records may not represent actual students.
- Career interests can change over time.
- The system cannot replace a professional academic adviser.
- Institutional admission requirements may differ between universities.
- Weighted scores may oversimplify complex student preferences.

Github link

https://github.com/kmmillares/Group21-CousePathwayAdvising

Course Pathway Advising Using Rule-Based Reasoning

Project Description

The Course Pathway Advising System is a Python-based academic decision-support system developed in Google Colab. It recommends suitable courses based on a student's completed courses, academic performance, interests, and target career.

The system uses prerequisite rules to determine which courses a student is currently eligible to take. It also identifies unmet prerequisites and generates a recommended course sequence. A weighted-scoring algorithm is used as a comparison method to rank possible courses according to career alignment, grades, interests, and prerequisite completion.

Features

- The system reads student records from a CSV file using Pandas.
- Knowledge base courses
- The system checks whether the student has completed the required courses.
- Rule based advising
- The system generates a sequence beginning with eligible courses and identifies prerequisite courses that must be completed before advanced courses.
- Courses are ranked using weighted scoring
- Explainable results of the systems

Algorithms

Main Algorithm: Rule-Based Reasoning

Rule-Based Reasoning is the primary algorithm of the Course Pathway Advising System. It uses predefined IF–THEN rules to determine which courses are appropriate for a student based on their completed courses, target career, and prerequisite requirements.

The system first retrieves the student's completed courses and target career from the CSV dataset. It then compares the student's information with the course catalog. For each course, the system checks whether the course is related to the student's selected career and whether all required prerequisites have been completed.

Eligible courses are organized into a recommended course sequence. Courses with incomplete prerequisites are not immediately recommended; instead, their missing prerequisite courses are reported so the student can complete them first.

Comparison Algorithm: Weighted Scoring

Weighted Scoring is used as a comparison algorithm for the Rule-Based Reasoning method. Instead of making decisions only through logical conditions, it assigns numerical values to different student and course characteristics.

Each course receives points based on four factors:

1. Career alignment
2. Academic performance
3. Interest alignment
4. Prerequisite completion

A course receives a higher score when it is related to the student's target career, matches the student's academic strengths, and corresponds to the student's interests. Courses with unmet prerequisites receive a penalty.

How the Scoring Works

The current scoring formula is:

Final Score = Career Alignment + Academic Performance + Interest Alignment - Prerequisite Penalty

1. Career Alignment

+40 points
A course receives 40 points when it belongs to the student's target career area.
if career in course_catalog[course]["career_area"]: score += 40

2. Mathematics Performance

For mathematics-related courses:
+20 points if mathematics grade is at least 85
For other courses:
+10 points if mathematics grade is at least 85

Mathematics-related courses include:

- Discrete Structures

- Data Structures And Alggorithm
- Algorithms
- Analytics Fundamentals
- Modeling and Simulation
- Image Processing
- Intelligent Systems

3. Programming Performance

For programming-related courses:
+20 points if programming grade is at least 85
For other courses:
+10 points if programming grade is at least 85

Programming-related courses include:

- Computer Programming
- Object Oriented Programming
- Structured Programming
- Programming Tools and Techniques
- Programming Languages
- Software Engineering
- Algorithms
- Mobile Programming
- Applications Development
- Operating Systems

4. Interest Alignment

Student interest is rated from 1 to 5.

Interest Rating	Points
1	4
2	8
3	12
4	16
5	20

The interest category depends on the course.
Examples:

- Programming courses use `interest_programming`.
- Data courses use `interest_data`.
- Networking courses use `interest_networking`.

- Security courses use `interest_security`.
- Business courses use `interest_business`.
- AI courses use `interest_ai`.

5. Prerequisite Penalty

-15 points for every unmet prerequisite
For example, if a course has two missing prerequisites:
Prerequisite penalty = 2 × -15 = -30 points

Example Calculation

Suppose a course has:

- Career alignment: 40 points
- Mathematics performance: 20 points
- Programming performance: 20 points
- Interest rating of 4: 16 points
- One unmet prerequisite: -15 points

Final Score = 40 + 20 + 20 + 16 - 15
Final Score = 81 points

Tie-Breaking and Ranking

Courses are ranked using this order:

1. Courses matching the target career.
2. Higher weighted score.
3. Alphabetical course order if scores are equal.

Courses that the student has already completed are excluded from the recommendation list.

Requirements and Installation

Requirements: Python, pandas, numpy, and scipy
Install dependencies in PowerShell:

```
py -m pip install pandas numpy scipy
py -m pip install openpyxl
py -m pip install streamlit pandas
```

Use the same Python launcher to run the script, so it uses the interpreter where the packages were installed

How to Use

1. Place .py in a project folder
2. Run .py
3. Streamlit will open and then upload the course_pathway_records.csv
4. Search a student profile (ex.S001)
5. Input the Following
    a. Career path
    b. Math grade
    c. Programming Grade
    d. Interest
6. Click Save Profile
7. View outputs then save results as csv

Important Notes

- Upload a CSV file using the required column names and format.
- Course names must exactly match the names in the course catalog, including spelling and capitalization.
- Separate multiple completed courses with commas.
- The target career must match one of the career paths listed in the system.
- A course with unmet prerequisites may appear in the results but should not be treated as immediately eligible.
- The recommended sequence is based only on the rules and information stored in the system.
- The weighted score is a supporting comparison and should not override prerequisite requirements.
- The results are advisory and should be reviewed with an academic adviser.
- The system does not guarantee admission, course availability, graduation completion, or employment.
- If the course catalog or institutional requirements change, the knowledge base must also be updated.

Troubleshooting

- **CSV will not load:** Check that the file is saved as `.csv`, uploaded correctly, and not corrupted.
- **Missing columns:** Verify that the CSV contains all required column names.
- **No recommendations:** Review the student's target career and unmet prerequisites.

- **Incorrect prerequisites:** Ensure course names are consistent in both the catalog and CSV file.